

## AWS Lab – EKS Scaling & Pipeline

Please create AWS environment as following:

- Connect to the aws console and assume your Role for privileged access
- Create a **VPC + 2 Private Subnets (delete the default VPC)**
- Configure **Routes** and configure limited **SG's Ports Opened**
- Create **EKS** cluster version 1.35
  - **Only Private Subnets Allowed, Node Groups are NOT ALLOWED!**
- Implement load Balancer Controller (**NGINX | | ALB or Both, as you wish**).
- Configure SSL Termination on the ALB, (use self-signed certificate)
- Deploy any CD tool on the cluster (argo cd, Octopus deploy, Bamboo, as you wish)
- Write an application in any language , wrap it with Helm3 and deploy on the EKS Cluster
- Surfing the frontend service, should show **"Hello Lab-Commit"** with 'X' as a String (X – app version number)  
\*\* Surfing only the **Secured URL** (Use Route53 Records & ACM)
- Create **RDS** MySQL/PostgreSQL and configure the backend service to **select a query from DB** and the frontend service will pull each X amount of time (5 seconds) the values from the backend service and present the value in the **HTTPS** response (Example "Hello Lab-commit 5" and etc..).
- The service should be **not exposed** to the Internet.
- Create a record in R53 for that application – Lab-commit-task.<your-hosted-zone>
- Deploy an EC2 Windows instance, connect to it via SSM (**no key-pair is allowed**) and surf your application via chrome.  
\*\* your service shouldn't be accessible publicly.
- Deploy on the EKS cluster some monitoring solution (you can pick anything, **CW agent is not allowed**) and expose it, so we can serf to it via the Windows instance and see the memory and cpu of your application.
- Deploy a dashboard with a 3<sup>rd</sup> party solution to show the numbers of the pods for the backend and the frontend.
- Write and deploy some "load generating" app/tool to generate requests to your application via Web.
- Implement some scaling tool on your environment so when the load will be more that 10 requests per second it will deploy addition pod for the frontend service, implement the load generator like this till 4 pods, and then show how the numbers of pods will decrease because there is no load anymore.
- Store all of your code in **Codecommit/GitHub**
- Create Pipeline with **CodePipeline** or SDK to deploy updated version of the frontend **application** (for example, will print **"Hello Lab-Commit v1.0.1"** etc..)
- Create above infrastructure via **Terraform**
- Once ready, please update the README.md file in the codecommit with the appropriate instructions to review the solution.